Diseño Técnico del Sitema - Villa in Chat

Nombre del fichero: VillaInChat_DiseñoTecnicoSistema.pdf

Fecha de esta versión: 03/05/2026

Versión: 1.0

Historial de revisiones

Fecha	Descripción	Autor
03/05/2026	Creación inicial del documento	Komodo Tech

ÍNDICE

1. Objetivo 3
2. Alcance 3
3. Stack tecnológico 3
4. Descripción general de la arquitectura 4
   4.1 Estructura de capas 4
   4.2 Estructura de directorios 4
   4.3 Navegación y routing 5
5. Modelo de datos 6
   5.1 Esquema de tablas 6
   5.2 Enumeraciones 8
   5.3 Tipos compuestos (TypeScript) 9
6. Módulo principal: Sistema de mensajería y menciones 9
   6.1 Descripción 9
   6.2 Clases y componentes relacionados 10
      ChatPanel (src/components/groups/ChatPanel.tsx) 10
      MentionInput (src/components/groups/MentionInput.tsx) 10
      mentionServices (src/services/mentionServices.ts) 10
      useMentions (src/hooks/useMentions.ts) 11
      NotificationProvider (src/providers/NotificationProvider.tsx) 11
      send-mention-notification (supabase/functions/send-mention-notification/index.ts) 11
      send-push (supabase/functions/send-push/index.ts) 11
      TriggerPostgreSQL (supabase/migrations/20260414211349_mention_trigger.sql) 11
   6.3 Relación entre componentes 12
7. Módulos secundarios 12
   7.1 Autenticación y sesión 12
   7.2 Grupos y canales 13
   7.3 Mensajes directos (DMs) 13
   7.4 Notificaciones push 13
8. Gestión de estado 14
9. Integración con Supabase 14

1. Objetivo

El objetivo de este documento es describir la arquitectura técnica de la aplicación VillaInChat, detallando las decisiones de diseño, la estructura de componentes, el modelo de datos y los flujos de comunicación entre capas. Sirve como referencia para desarrolladores que necesiten implementar, mantener o ampliar cualquier funcionalidad de la plataforma.

2. Alcance

Esta guía está dirigida a desarrolladores que trabajen en el frontend (React Native / Expo), en la lógica de backend (Supabase, Edge Functions) o en la integración entre ambas capas. Cubre desde el modelo de datos hasta los flujos de datos en tiempo real.

3. Stack tecnológico

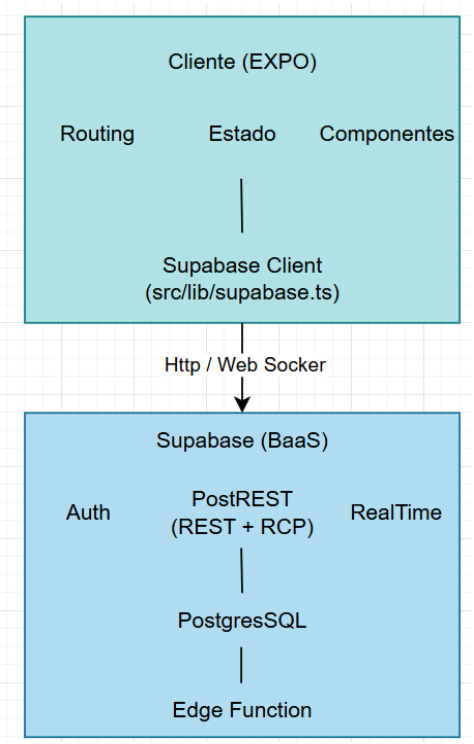
Capa	Tecnología	Versión
Framework principal	React Native + Expo	0.81.5 / 54.0
Routing	expo-router (file-based)	6.0
Lenguaje	TypeScript	5.9
Estado global	Zustand	5.0
Estado contextual	React Context API	–
Formularios	react-hook-form + Zod	7.71 / 4.3
Animaciones	react-native-reanimated	4.1
Gestos	react-native-gesture-handler	2.28
Modales	@gorhom/bottom-sheet	5.2
Backend (BaaS)	Supabase (Auth, PostgreSQL, Realtime)	–
Base de datos	PostgreSQL	14.1
Edge Functions	Deno (TypeScript)	–

Notificaciones push	Expo Push Service	–
Almacenamiento local	AsyncStorage / localStorage (web)	2.2
Build / Despliegue	EAS Build (móvil), Vercel (web)	–

4. Descripción general de la arquitectura

VillaInChat sigue una arquitectura cliente-servidor basada en BaaS, donde Supabase actúa como backend completo (autenticación, base de datos relacional, realtime y funciones serverless). El cliente es una aplicación React Native multiplataforma (Android, iOS, Web).

4.1 Estructura de capas



4.2 Estructura de directorios

```
villaInChat/
├── app/              → Rutas (expo-router, file-based)
│   ├── _layout.tsx     → Layout raíz: providers + Stack navigator
│   ├── index.tsx       → Pantalla de carga inicial
```

```
│   ├── (auth)/          → Rutas de autenticación
│   ├── (tabs)/          → Rutas principales (Tab navigator)
│   ├── groups/[id].tsx    → Detalle de grupo (canales + chat)
│   ├── rooms/[id].tsx     → Sala de chat o DM
│   └── user/[id].tsx      → Perfil de usuario
│
├── src/
│   ├── actions/         → Mutaciones RPC con validación Zod
│   ├── components/        → Componentes por dominio
│   ├── hooks/           → Hooks reutilizables
│   ├── providers/         → Contextos globales
│   ├── services/         → Lógica de negocio atómica
│   ├── stores/          → Zustand stores
│   ├── schemas/          → Esquemas de validación Zod
│   ├── lib/             → Clientes singleton
│   ├── types/           → Tipos TypeScript
│   ├── themes/           → Design system
│   └── storage/         → Wrappers de AsyncStorage
│
└── supabase/
    ├── functions/        → Edge Functions Deno
    └── migrations/       → SQL y triggers
```

4.3 Navegación y routing

Expo-router implementa routing basado en el sistema de ficheros. La jerarquía es:

```
_layout.tsx (Stack raíz)
│
├── AuthProvider → redirige a /SignIn o /(tabs)/home
│
├── (auth)/  →  SignIn, SignUp
│
└── (tabs)/  →  Tab Navigator
    ├── home.tsx           → Lista de grupos
    ├── privateChatRooms.tsx → Lista de DMs
    ├── notifications.tsx    → Menciones recibidas
    └── settings.tsx        → Preferencias
   + Rutas dinámicas:
    ├── groups/[id].tsx     → Detalle de grupo
    ├── rooms/[id].tsx      → Vista de chat
    └── user/[id].tsx       → Perfil de usuario
```

5. Modelo de datos

5.1 Esquema de tablas

user_profile

Columna	Tipo	Descripción
user_id	uuid (PK)	Referencia al usuario de Supabase Auth
username	text	Nombre de usuario visible
user_role	UserType	Rol global: ADMIN, TEACHER, STUDENT
push_token	text | null	Token Expo Push para notificaciones
notifications_enabled	boolean	Si el usuario recibe notificaciones regulares
grade	text | null	Curso del alumno
phone	text | null	Teléfono

group_room

Columna	Tipo	Descripción
group_id	uuid (PK)	Identificador del grupo
group_name	text	Nombre del grupo
created_at	timestamp	Fecha de creación

group_members

Columna	Tipo	Descripción
FK_group_id	uuid (FK)	Grupo al que pertenece

FK_user_id	uuid (FK)	Usuario miembro
user_role	ChatPrivileges	Rol: ADMIN, CLASS_REP, STUDENT
created_at	timestamp	Fecha de incorporación

chat_room

Columna	Tipo	Descripción
chat_id	uuid (PK)	Identificador del canal/chat
name	text	Nombre del canal
chat_type	ChatType	PRIVATE, PUBLIC, ANNOUNCEMENTS
FK_group_id	uuid | null	null si es DM

Un DM es un chat_room con chat_type = PRIVATE y FK_group_id = NULL.

chat_members

Columna	Tipo	Descripción
member_id	uuid (PK)	Identificador de membresía
FK_chat_id	uuid (FK)	Sala a la que pertenece
FK_user_id	uuid (FK)	Usuario miembro

messages

Columna	Tipo	Descripción
id	bigint (PK)	Identificador del mensaje

content	text	Contenido del mensaje
FK_chat_id	uuid (FK)	Sala donde se envió
FK_author_id	uuid (FK)	Autor del mensaje
created_at	timestamp	Fecha de envío
read	boolean	Estado de lectura

mentions

Columna	Tipo	Descripción
mention_id	uuid (PK)	Identificador de la mención
type	MentionType	USER (directo) o EVERYONE
FK_message_id	bigint (FK)	Mensaje que contiene la mención
FK_chat_id	uuid (FK)	Sala donde ocurrió
FK_sender_id	uuid (FK)	Usuario que mencionó
FK_recipent_id	uuid | null	Usuario mencionado (null si EVERYONE)
read	boolean	Si el receptor la ha leído
created_at	timestamp	Fecha de la mención

5.2 Enumeraciones

Enum	Valores
ChatType	PRIVATE, PUBLIC, ANNOUNCEMENTS

ChatPrivileges	ADMIN, CLASS_REP, STUDENT
UserType	ADMIN, TEACHER, STUDENT
MentionType	USER, EVERYONE

5.3 Tipos compuestos (TypeScript)

Definidos en src/types/index.ts:

```
// Aliases de tablas
type Chat        = Database['public']['Tables']['chat_room']['Row']
type Messages     = Database['public']['Tables']['messages']['Row']
type Users        = Database['public']['Tables']['user_profile']['Row']
type Mention      = Database['public']['Tables']['mentions']['Row']

// Tipos con relaciones
type MessageWithUser    = Messages & { user: Users }
type MentionWithDetails = Mention & {
   sender:  { username: string }
   chat:    { name: string | null }
   message: { content: string }
}
```

6. Módulo principal: Sistema de mensajería y menciones

6.1 Descripción

El sistema de mensajería es el núcleo funcional de VillaInChat. Permite enviar mensajes en tiempo real dentro de canales de grupo o DMs, con soporte para menciones directas **(@username)** y globales **(@everyone)**. Las menciones disparan notificaciones push a través de un trigger de base de datos y una Edge Function.

6.2 Clases y componentes relacionados

ChatPanel (src/components/groups/ChatPanel.tsx)

Componente principal de visualización de chat. Gestiona la carga inicial de mensajes (últimos 50, orden descendente), la suscripción Realtime para nuevos mensajes, el envío con parsing de menciones y el renderizado de burbujas y separadores de fecha.

Props: chatId, chatName, onBack, groupMembers?, chatType?, userGroupRole?

MentionInput (src/components/groups/MentionInput.tsx)

Input de texto con autocompletado. Detecta el prefijo @ y muestra un dropdown con miembros del grupo y la opción @everyone. Al seleccionar, inserta la mención en la posición correcta del cursor.

Props: value, onChangeText, groupMembers, onSubmitEditing?

mentionServices (src/services/mentionServices.ts)

Función	Descripción
parseMentions(content, members)	Parsea el texto con regex, devuelve array de menciones
insertMentions(messageId, chatId, senderId, mentions)	Inserta menciones en la tabla mentions
fetchMentionsForUser(userId)	Obtiene todas las menciones del usuario
markMentionAsRead(mentionId)	Marca una mención como leída
markAllMentionsAsRead(userId)	Marca todas las menciones como leídas

useMentions (src/hooks/useMentions.ts)

Hook que encapsula el ciclo de vida de las menciones: carga inicial, suscripción Realtime a la tabla mentions filtrada por **FK_recipent_id**, y exposición de mentions, **mentionUnreadCount**, **markAsRead**, **markAllRead**.

NotificationProvider (src/providers/NotificationProvider.tsx)

Provider que compone **usePushToken**, **useNotification** y **useMentions.** Expone **NotificationContext** con el estado global de notificaciones y menciones.

send-mention-notification (supabase/functions/send-mention-notification/index.ts)

Edge Function Deno activada por trigger. Determina los receptores (1 usuario si **USER**, todos los miembros si **EVERYONE**), obtiene sus **push_token** sin filtrar **notifications_enabled**, y envía el batch a la Expo Push API.

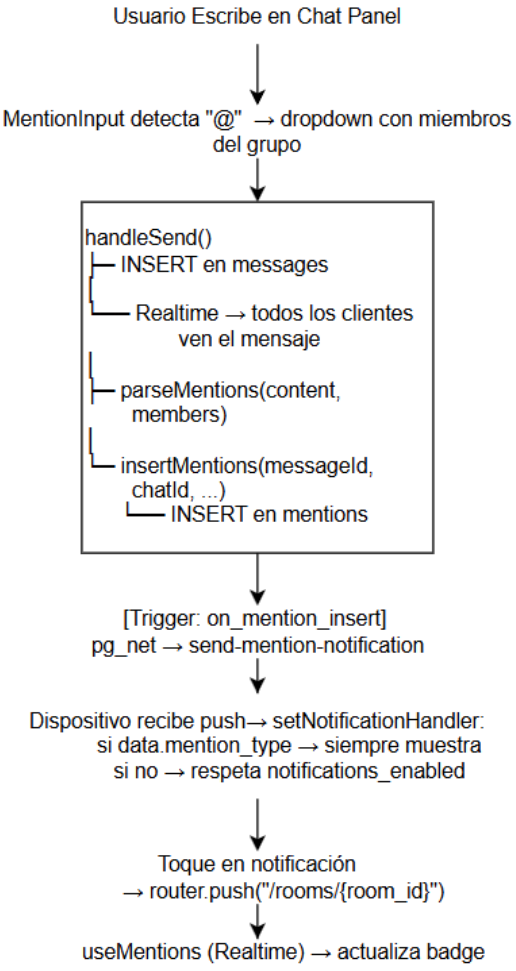
send-push (supabase/functions/send-push/index.ts)

Edge Function para mensajes regulares. Filtra por ***notifications_enabled = true*** y omite el proceso si el mensaje contiene ***@everyone.***

TriggerPostgreSQL (supabase/migrations/20260414211349_mention_trigger.sql)

Define **on_mention_insert** sobre la tabla mentions. Usa **pg_net** para llamar a send-mention-notification en cada **INSERT**.

6.3 Relación entre componentes



7. Módulos secundarios

7.1 Autenticación y sesión

Ficheros:**src/providers/AuthProvider.tsx,src/lib/supabase.ts, app/(auth)/SignIn.tsx**

El cliente Supabase es un singleton con adaptador de almacenamiento personalizado (AsyncStorage en móvil, localStorage en web). **AuthProvider** escucha **onAuthStateChange**: sesión activa → redirige a **/(tabs)/home**; sesión destruida → redirige a **/(auth)/SignIn**.

7.2 Grupos y canales

Ficheros:**app/(tabs)/home.tsx, app/groups/[id].tsx, src/actions/rooms.ts**

Un grupo (**group_room**) contiene múltiples canales (**chat_room** con **FK_group_id**). Los canales **PUBLIC** y **ANNOUNCEMENTS** se asignan automáticamente a los miembros al unirse. Los canales **PRIVATE** solo incluyen a miembros invitados.

groups/[id].tsx implementa un layout de dos paneles animado con Reanimated: panel izquierdo (lista de canales) y panel derecho (**ChatPanel**). En móvil se navega con gestos; en escritorio ambos son visibles simultáneamente.

Las creaciones complejas usan RPC functions con **SECURITY DEFINER** para operar fuera de las restricciones RLS:

Función RPC	Descripción
create_group(p_name, p_member_ids)	Crea grupo y añade miembros
create_group_chat(p_group_id, p_name, p_chat_type)	Crea canal y suscribe miembros
create_dm(p_receiver_id, p_message)	Crea chat privado con mensaje inicial

7.3 Mensajes directos (DMs)

Ficheros:***app/(tabs)/privateChatRooms.tsx,src/components/users/UserBottomSheet.tsx***

Un DM es un **chat_room** con **chat_type = PRIVATE** y **FK_group_id = NULL**. Antes de crear uno, **findExistingDM()** verifica si ya existe conversación entre los dos usuarios para evitar duplicados. La creación desde la UI pasa por **UserBottomSheet**.

7.4 Notificaciones push

Ficheros:***src/lib/notifications.ts,src/hooks/usePushToken.ts,supabase/functions/send-push/***

Al iniciar sesión, **usePushToken** registra el token del dispositivo en **user_profile**. Al registrar, limpia ese token de cualquier otro usuario (previene colisiones entre cuentas en el mismo dispositivo). El cliente gestiona la visualización en foreground mediante **setNotificationHandler**, que distingue menciones (siempre visibles) de mensajes regulares (respeta **notifications_enabled**).

8. Gestión de estado

El estado se gestiona en tres niveles:

Zustand — Estado global persistido (src/stores/useSettingsStore.tsx)
Gestiona theme (light/dark/system) y **welcomeShown**. Persiste en AsyncStorage/localStorage con hidratación asíncrona.

Context API — Estado contextual

Provider	Expone
AuthProvider	session, loading
NotificationProvider	mentions, mentionUnreadCount, markAsRead, markAllRead
UserSheetProvider	openUserSheet(user)

useState local
Mensajes, estados de carga, formularios. Las suscripciones Realtime se crean en **useEffect** con cleanup en el unmount.

9. Integración con Supabase

Cliente singleton
Todas las capas importan el mismo cliente:

```
import { supabase } from "@/src/lib/supabase"
```

Row Level Security (RLS)

Todas las tablas tienen RLS activado. Las políticas usan funciones auxiliares **SECURITY DEFINER** para evitar recursión infinita:

Función	Verifica
rls_user_in_group(group_id)	Si el usuario pertenece al grupo
rls_user_is_group_admin(group_id)	Si el usuario es ADMIN del grupo
rls_user_in_chat(chat_id)	Si el usuario pertenece al canal
rls_user_can_write_in_chat(chat_id)	Si el usuario puede escribir (tipo de canal + rol)

Consultas Realtime

Los canales usan nombres únicos por mount (**chat_panel:{chatId}:{Date.now()}**) para evitar conflictos con el doble render de React 18 Strict Mode. Se limpian con **supabase.removeChannel()** al desmontar.

Variables de entorno

Variable	Uso
EXPO_PUBLIC_SUPABASE_URL	URL del proyecto Supabase
EXPO_PUBLIC_SUPABASE_ANON_KEY	Clave pública (anon key)

Para EAS Build: eas env:create --scope project. Para Vercel: dashboard de Vercel. app.json no interpola variables de entorno — para configuración dinámica usar app.config.js.